Examen oral SO

Stiva software

Ce este un apel de sistem?

Un apel de sistem este o cerere a unei aplicații, care rulează în user-space în mod neprivilegiat, către sistemul de operare pentru efectuarea unei operații privilegiate, care va rula în kernel space. Este singurul mod prin care software-ul poate genera întreruperi.

De ce sunt necesare apeluri de sistem?

Pentru ca doar sistemul de operare să aibă acces în mod direct la resursele sistemului și pentru a permite acestuia sa controleze alocarea resurselor și modul în care acestea pot fi accesate.

Ce avantaj / dezavantaje au apelurile de sistem?

Un avantaj este ca la un apel de sistem noi apelăm cod scris de altcineva, nu trebuie sa îl scriem noi. Fără sistemul de operare, și implicit apelurile de sistem, ne-ar fi mult mai greu sa folosim calculatorul. Alt avantaj este securitatea sistemului, userii nu pot strica sistemul.
Un dezavantaj este overhead-ul (trecerea din user space în kernel space) și eventual cod mai mult (mai multe instrucțiuni) decât ce ne este nouă necesar într-o anumită aplicație (adică noi am scrie mai puține instrucțiuni pentru a face același lucru ca apelul de sistem).

Ce înseamnă user/application mode/space (mod neprivilegiat)? Ce înseamnă kernel/supervisor mode/space (mod privilegiat)?

User mode este modul în care rulează, în general, un proces. Unui proces îi este atribuit un spațiu virtual de adrese care poate fi accesat doar de acel proces.
Kernel mode este modul în care se rulează sistemul de operare (dar nu partea de aplicații din SO, ci mai degrabă kernelul). În mod privilegiat ai acces la un set mai mare de instrucțiuni/registre decât în modul neprivilegiat.

Cum se realizează tranziția din mod neprivilegiat în mod privilegiat?

Sistemul de operare expune o interfață (system call interface, system API) prin care aplicațiile solicită acțiuni privilegiate sistemului de operare.

Ce se întâmplă în momentul tranziției în mod privilegiat? Cum se/Cine asigură (enforcement) existența modului privilegiat?

Aplicațiile solicită acțiuni privilegiate sistemului de operare. Sistemul de operare verifică dacă aplicația are permisiunile necesare, realizează acțiunea cerută și întoarce rezultatul. Sistemul de operare are rolul unui furnizor de servicii privilegiate pentru aplicații.

Ce este o bibliotecă?

O biblioteca este o colectie de functii precompilate.

Care este diferența dintre o aplicație și o bibliotecă?

Aplicația este un program software complet, care poate fi utilizat direct de utilizator pentru a îndeplini o anumită sarcină. O bibliotecă, pe de altă parte, este un set de coduri prescrise care poate fi utilizat în aplicații pentru a realiza anumite funcții sau a rezolva anumite probleme. Aplicațiile sunt construite utilizând biblioteci, dar ele sunt complete și independente în sine.

Care este asocierea apel de bibliotecă / apel de sistem?

Un apel de biblioteca presupune zero, unul, sau mai multe apeluri de sistem.
De exemplu, pentru un scanf, se va face un read în spate dacă datele ce trebuie citite nu sunt deja în bufferul menținut de libc de la un read anterior. Altfel, nu se va face read.

Ce este entry point-ul într-un executabil?

Este o adresă specifică în memoria computerului unde se află primul cod care este executat când se lansează un program.

Care este rolul bibliotecii standard C (libc)?

Ofera aplicatiilor wrappere peste apeluri de sistem pentru a putea fi apelate ca funcțiile dintr-o biblioteca. Acest lucru face codul scris cu functiile din libc portabil peste mai multe interfețe de sistem/platforme.

Ce acțiuni se pot executa doar în mod privilegiat?

I/O, mapari și alocări de memorie, managementul timerelor și al întreruperilor, comunicarea cu alte procese, configurarea de permisiuni, pornirea/oprirea aplicațiilor

Ce operații / instrucțiuni low-level (ISA) se pot executa doar în mod privilegiat?

- Accesarea memoriei sistemului sau a registrelor speciale
- Maparea memoriei
- Configurarea sau controlarea componentelor hardware (cum ar fi unitățile de intrare/ieșire sau interrupțiile)
- Modificarea regulilor de securitate sau de gestionare a excepțiilor
- Executarea instrucțiunilor privind administrarea resurselor sau a protecției sistemului.

Ce este un sistem de operare monolitic?

Un sistem de operare monolitic este un sistem în care user services și kernel services sunt implementate în același spațiu de adrese (motiv pentru care este eficient). Este mai puțin flexibil, pentru ca atunci cand se face o modificarea asupra unei componente, trebuie schimbat tot. Este un sistem mai mare, intrucat kernelul este mai mare. Dacă un serviciu pica, pica toate. Toate functiile SO-ului se realizeaza prin apeluri de sistem.

Ce este un sistem de operare de tip microkernel?

Un sistem de operare de tip microkernel are user services si kernel services implementate în locuri diferite din memorie (motiv pt care este mai lent, serviciile fiind nevoite sa comunice între ele). Se poate extinde ușor și este un SO mic, kernelul fiind mic. Dacă un serviciu, fie el kernel sau user, pica, nu afectează alte servicii.

Care sunt avantajele unui sistem de operare monolitic?

- Execuție mai rapidă
- Necesita mai puțin cod

Care sunt avantajele unui sistem de operare de tip microkernel?

- Se extinde mai ușor
- Dacă un serviciu crapă, nu afectează alte servicii.

Care tip de sistem de operare are mai multe apeluri de sistem?

Cel de tip microkernel pentru ca sunt mult mai multe tranziții user kernel pentru a permite componentelor software care rulează în user mode să interacționeze.

Care este avantajul folosirii mașinilor virtuale din perspectiva securității?

- Izolare: Mașinile virtuale oferă o izolare a sistemelor gazdă și invitate, prevenind răspândirea eventualelor amenințări sau infectări de la sistemul invitat la sistemul gazdă.
- Testare sigură: Mașinile virtuale permit testarea sigură a unui software sau a unui sistem operativ fără a afecta sistemul gazdă sau a expune acesta la riscuri.
- Monitorizare ușoară: Se poate monitoriza activitatea sistemului invitat și se pot lua măsuri de securitate adecvate în cazul în care se detectează comportamente neregulate sau amenințări.
- Restricții de acces: Accesul la resursele sistemului gazdă poate fi restricționat prin configurarea mașinii virtuale, ceea ce îmbunătățește securitatea sistemului în ansamblu.

Ce este o bibliotecă statică? Ce este o bibliotecă dinamică?

O biblioteca statică (.a pe Linux, .lib pe Windows) este o colecție de funcții linkate la compile time. Biblioteca statică face parte din executabilul obtinut.
O biblioteca dinamica(.so pe Linux, .dll pe Windows) este o colecție de funcții linkate la load time.

Când preferăm folosirea static linking, respectiv dynamic linking? Cu ce diferă un executabil dinamic de un executabil static?

Static linking se prefera atunci când:
- se dorește o distribuție mai ușoară a aplicației, fără a fi necesare bibliotecile externe
- se dorește control mai mare asupra versiunii bibliotecilor folosite
- se dorește un control mai mare asupra performanței, deoarece bibliotecile sunt încărcate în memoria aplicației de la început

Dynamic linking se prefera atunci când:
- se dorește utilizarea unei singure copii a bibliotecii de către mai multe aplicații
- se dorește posibilitatea de a actualiza biblioteca fără a recompila aplicațiile care o folosesc.

Un executabil dinamic conține referințe către biblioteci externe care sunt încărcate la rulare, în timp ce un executabil static conține toate bibliotecile necesare încorporate direct în fișierul executabil. Astfel:
- Executabilul dinamic are o dimensiune mai mica deoarece bibliotecile externe nu sunt incluse, dar are nevoie de acestea pentru a funcționa.
- Executabilul static poate fi distribuit mai ușor, deoarece nu are nevoie de bibliotecile externe, dar poate avea dimensiune mai mare și poate fi mai greu de optimizat.

Dați exemplu de apel de sistem blocant.

- read() - se blocheaza pana poate fi citit cel putin un byte
- write() - se blocheaza pana s-a putut scrie intregul buffer

Dați exemplu de apel de sistem neblocant.

send() - nu se blochează dacă nu se încearcă trimiterea mai multor bytes decat încap în buffer

De ce, în general, o aplicație trebuie să execute un apel de sistem pentru a accesa un dispozitiv hardware? De ce NU poate accesa direct dispozitivul hardware?

Pentru ca trebuie sa fie într-un mod privilegiat (apelul de sistem executat cu succes asigura aceste privilegii).
Pentru ca trebuie sa fie mapat in memorie pentru a putea realiza comunicarea.

Ce înțelegem prin overhead spațial și overhead temporal?

Overhead spațial se referă la spațiul suplimentar necesar pentru stocarea informațiilor suplimentare în memorie, cum ar fi structuri de date, metadate sau informații de administrare.
Overhead temporal se referă la costul adițional de timp necesar pentru a efectua o acțiune sau o operație, cum ar fi prelucrarea datelor sau alocarea de memorie.

Dați exemplu de mecanism / funcție care reduce overhead-ul spațial și unul care reduce overhead-ul temporal.

Compresia datelor: utilizarea algoritmilor de compresie pentru a reduce dimensiunea informațiilor stocate în memorie. (overhead spațial)
Optimizarea algoritmilor: utilizarea unor algoritmi mai eficienți pentru a reduce timpul necesar pentru a efectua o activitate. (overhead temporal)

Ce înseamnă double buffering? În ce situație concretă (mecanism/funcție) apare?

Double buffering = tehnica prin care 2 buffere sunt folosite pentru creșterea performanței. Astfel procesarea unor date se poate face independent: în timp ce se lucrează cu datele dintr-un buffer, se pot pregăti date in celălalt buffer.

Exemplu: SO vrea sa scrie date către disk care e lent și ca sa nu stea după el, scrie într-un buffer și le ia în ritmul lui.

Ce se întâmplă când există o eroare critică (de tip Segmentation fault) la nivelul sistemului de operare?

Hardware-ul notifica sistemul de operare și se apelează o rutina din sistemul de operare. Sistemul de operare trebuie sa trateze eroarea.

Dați exemplu de un apel de bibliotecă standard C care nu cauzează apel de sistem.

Un exemplu de apel de bibliotecă standard C care nu cauzează un apel de sistem ar fi "strlen". Funcția "strlen" determină lungimea unei șiruri de caractere (în număr de caractere), și poate fi inclusă în codul aplicației fără a apela sistemul.

Dați exemplu de un apel de bibliotecă standard C care cauzează apel de sistem.

Un exemplu de apel de bibliotecă standard C care cauzează un apel de sistem ar fi "fopen". Funcția "fopen" deschide un fișier pentru citire sau scriere și poate cauza un apel de sistem pentru a accesa sistemul de fișiere.

Care este avantajul și dezavantajul existenței unei stive software la nivelul unui sistem de calcul?

Avantaje: structură de date eficientă, simplitate: stiva software este simplă și ușor de implementat și înțeles, folosirea resurselor
Dezavantaje: limitarea dimensiunii, debugging dificil, performanță scăzută

Care este avantajul și dezavantajul folosirii unui limbaj de programare high-level (precum Java, Python, Lua)?

Avantaje: ușor de învățat și utilizat, portabilitate. biblioteci bogate
Dezavantaje: performanță mai slabă, resurse mai mari, abstracție excesivă

Care este avantajul și dezavantajul folosirii unui limbaj de programare low-level (precum C, C++, Rust, D)?

Avantaje: performanță mai bună, control mai fin, capacitatea de a construi sisteme de bază
Dezavantaje: dificultăți în învățare și utilizare, resurse hardware mai mari, abstracție scăzută

Date

Cum asigură sistemul de operare separația între procese?

Sistemul de operare asigură separarea între procese prin mecanisme precum: alocarea resurselor (ex. memorie, CPU), spațiul de adrese virtuale, controlul accesului la fișiere și alte resurse, planificarea proceselor.

Ce înseamnă mecanismul de memorie virtuală?

Mecanismul de memorie virtuală este o tehnică utilizată de sistemele de operare pentru a administra memoria computerului, care permite unui proces să acceseze cantități mai mari de memorie decât cea disponibilă fizic. Funcționează prin maparea zonei de memorie virtuală a unui proces în memorie fizică, astfel încât fiecare proces să aibă acces la o cantitate suficientă de memorie, fără a interfera cu alte procese. Acest mecanism permite, de asemenea, transferul temporar de pagini între memoria fizică și disc, ceea ce poate îmbunătăți eficiența utilizării memoriei.

Ce reprezintă spațiul virtual de adrese al unui proces?

Spațiul virtual de adrese al unui proces reprezintă un set de adrese logice asociate procesului, care se traduc în adrese fizice în memoria principală prin intermediul mecanismului de memorie virtuală. Acest spațiu de adrese oferă un context separat pentru fiecare proces, permițând accesul la resursele de memorie necesare fără a afecta alte procese. Sistemul de operare gestionează traducerea adreselor virtuale în adrese fizice, asigurând astfel separarea fizică a resurselor de memorie între procese.

Cu ce diferă zona de date de zona de cod în spațiul virtual de adrese al unui proces?

Zona de date conține datele utilizate de un proces, inclusiv variabilele și alte structuri de date. Zona de cod conține instrucțiunile ce definesc comportamentul procesului. În spațiul de adrese virtual, zona de cod este protejată împotriva modificărilor accidental sau intenționate, în timp ce zona de date este accesibilă pentru citire și scriere.

Ce permisiuni are zona .text/.data/.rodata/.bss/de stivă/de heap?

- .text: Este zona de cod și are permisiuni de execuție (EXEC).
- .data: Este zona de date inițializate și are permisiuni de citire și scriere (READ/WRITE).
- .rodata: Este zona de date nemodificabile și are permisiuni doar de citire (READ).

- .bss: Este zona de date neinițializate și are permisiuni de citire și scriere (READ/WRITE).
- stivă: Este un spațiu de memorie utilizat pentru stocarea temporară a informațiilor (cum ar fi variabile locale și adrese de returnare) și are permisiuni de citire și scriere (READ/WRITE).
- heap: Este un spațiu de memorie dinamic alocat la runtime și are permisiuni de citire și scriere (READ/WRITE).

Ce este paginarea memoriei?

Pentru a imparti memoria eficient, tinand cont ca diferite procese au nevoie de dimensiune de memorie diferită, procesele pot cere mai multă memorie la runtime și pentru ca permisiunile la diferite zone din aceasta memorie trebuie sa fie diferite se dorește alocarea de bucăți, în loc de doar o zona mare. Pagina este o bucata de dimensiune fixa (de obicei 4kb). Paginarea memoriei înseamnă impartirea memoriei în pagini. Paginarea e cea care genereaza fragmentarea internă.

Ce rol are tabela de pagini?

Tabela de pagini ajută la implementarea protecției adreselor, asigurând că un proces nu poate accesa accidental sau intenționat zonele de memorie ale altor procese sau ale sistemului de operare.

Ce este și ce rol are MMU (Memory Management Unit)?

MMU (Unitatea de Management a Memoriei) este o componentă hardware a unui sistem care se ocupă de maparea adreselor virtuale ale unui proces la adresele fizice ale memoriei.
Rolul MMU este de a traduce adresele virtuale utilizate de un proces în adrese fizice ale memoriei, astfel încât procesul să poată accesa datele stocate în memoria fizică. MMU validează, de asemenea, adresele accesate de proces, asigurând că acestea sunt valide și că procesul nu poate accesa zonele de memorie interzise (cum ar fi memoria altor procese sau a sistemului de operare).

Ce rol are TLB?

TLB are rolul de a asocia adrese virtuale cu adrese fizice de memorie, accelerează accesul la pagini de memorie şi reduce numărul de operaţii de căutare necesare pentru a localiza pagina dorită.

Care este ordinul de mărime al numărului de intrări ale TLB?

Ordinul de mărime al numărului de intrări în TLB poate fi aproximativ de ordinul zecilor sau sutelor. În general, cu cât TLB-ul este mai mare, cu atât mai multe adrese pot fi stocate în cache, ceea ce poate conduce la o performanță mai bună. Cu toate acestea, dimensiunea TLB-ului poate avea un impact asupra dimensiunii cache-ului și asupra consumului de energie. Prin urmare, dimensiunea TLB-ului este optimizată în funcție de cerințele specifice ale sistemului.

Ce conține o intrare în tabela de pagini?

O intrare în tabela de pagini conține informații despre maparea dintre o adresă virtuală și o adresă fizică în memoria principală. Această mapare este utilizată de MMU (Unitatea de Management a Memoriei) pentru a traduce adresele virtuale în adrese fizice reale atunci când un proces accesează memoria.

Ce înseamnă tabelă de pagini multi-nivel (ierarhică)? De ce este utilă?

Tabela de pagini multi-nivel (ierarhică) se referă la o structură de gestionare a memoriei în care fiecare nivel conține referințe la pagini sau la alte nivele de pagini, până când se ajunge la pagina finală. Acest lucru permite sistemului să mapeze adrese virtuale foarte mari în adrese fizice prin utilizarea mai multor niveluri de referințe.
În general, tabela de pagini multi-nivel (ierarhică) este utilă pentru gestionarea eficientă a memoriei în sistem cu adrese virtuale mari, prin reducerea dimensiunii necesare tabelei de pagini și prin îmbunătățirea performanței de mapare a adreselor virtuale în adrese fizice.

Când are loc un TLB miss?

Un TLB miss are loc atunci când CPU-ul caută o intrare în TLB și nu o găsește. Acest lucru înseamnă că adresa virtuală nu mai este ținută în cache-ul TLB și trebuie să fie accesată din tabela de pagini din memoria principală.

De ce se golește TLB-ul (TLB flush) la schimbare de context?

Se golește TLB-ul (TLB flush) la schimbare de context deoarece fiecare proces sau task are propria sa adresă virtuală și fizică și trebuie să existe o corespondență clară între acestea. Când se schimbă contextul (de la un proces la altul), adresele virtuale utilizate de procesul anterior nu mai sunt valide pentru procesul curent.

De ce nu este nevoie de TLB flush la schimbarea de context între două thread-uri ale aceluiași proces?

La schimbarea de context între două thread-uri ale aceluiași proces, nu este nevoie de un flush TLB (golirea cache-ului TLB) deoarece toate thread-urile din acel proces folosesc aceeași adresă virtuală și aceeași tabelă de pagini.

Ce înseamnă mecanismul de copy-on-write?

Copy-on-write (CoW) este un mecanism de optimizare a performanței care se bazează pe ideea că multe procese pot compartimenta aceleași pagini de memorie înainte de a fi necesară o modificare. Astfel, atunci când un proces are nevoie să modifice o pagină de memorie compartimentată, se creează o copie a paginii, astfel încât fiecare proces să aibă propria sa versiune a acesteia.

Dați exemple de situații în care are loc mecanismul de copy-on-write.

Copy-on-write (CoW) poate fi utilizat în mai multe situații, inclusiv: crearea de procese, crearea de thread-uri, gestionarea memoriei, implementarea de sisteme de fișiere, implementarea de biblioteci compartimentate.

Cu ce apel de sistem asociem copy-on-write?

Copy-on-write (CoW) este adesea asociat cu apelul de sistem "fork". Acest apel de sistem poate fi utilizat pentru a crea un proces copil prin utilizarea mecanismului CoW. Când un proces apelează "fork", se creează un nou proces copil care împarte aceleași resurse cu procesul părinte, inclusiv spațiul de adrese virtual și tabela de pagini. Atunci când un proces copil modifică conținutul unei pagini, acea pagină este clonată (copiată) în memorie și modificările sunt efectuate pe copie, astfel încât procesul părinte să nu fie afectat.

Când se duplică o pagină marcată copy-on-write?

O pagină marcată cu mecanismul copy-on-write (CoW) se duplică atunci când este modificată de către un proces. Înainte de a fi modificată, pagina este marcată ca fiind partajată între mai multe procese. Atunci când un proces încearcă să modifice conținutul unei pagini partajate, sistemul de operare duplică acea pagină într-o locație separată în memorie, astfel încât modificările să nu afecteze alte procese care împart aceeași pagină. După ce a fost duplicată, acea pagină nu mai este marcată CoW și devine unica responsabilitate a procesului care a efectuat modificarea.

Cine detectează un acces de scriere într-o pagină marcată copy-on-write?

Detecția accesului de scriere într-o pagină marcată cu mecanismul copy-on-write (CoW) este realizată de către sistemul de operare. Acesta monitorizează accesul la pagini în memoria fizică și detectează orice încercare de a modifica conținutul unei pagini marcate cu CoW. Când se detectează un astfel de acces, sistemul de operare duplică acea pagină într-o locație separată în memorie și marchează pagina originală ca fiind protejată împotriva modificărilor ulterioare. Acest proces este transparent pentru procesele de sistem și se execută automat.

Ce înseamnă demand paging?

Demand paging este un mecanism de management al memoriei utilizat în sistemele de operare care permite unui proces să acceseze mai multe pagini decât sunt disponibile în memoria fizică a sistemului. Acest lucru se face prin încărcarea paginilor în memoria fizică doar atunci când sunt necesare, ceea ce se numește "paging on demand". În caz contrar, paginile care nu sunt accesate sunt stocate pe disc, iar acest spațiu de stocare poate fi utilizat pentru a stoca alte pagini necesare altor procese. Prin această tehnică, sistemul poate gestiona eficient memoria disponibilă, permitând unui număr mai mare de procese să ruleze simultan fără a fi necesară o cantitate suplimentară de memorie fizică.

În ce situație apare page fault fără a cauza segmentation fault?

Un page fault poate apărea fără a cauza un segmentation fault în următoarele situații:

- Acces la o adresă nealocată din heap: atunci când un program accesează o adresă din zona de heap care nu a fost alocată încă, se produce un page fault, dar nu și un segmentation fault.
- Acces la o pagină marcată copy-on-write: atunci când un proces încearcă să scrie într-o pagină marcată copy-on-write, se produce un page fault, dar nu și un segmentation fault.
- Acces la o pagină neîncărcată din disc: atunci când un proces accesează o pagină care nu este încă în memoria fizică, dar care există pe disc, se produce un page fault, dar nu și un segmentation fault.

Ce rol are spațiul de swap?

Rolul principal al spațiului de swap este de a asigura continuitatea sistemului de operare atunci când nu mai există suficientă memorie RAM disponibilă pentru a gestiona toate procesele în funcționare. În acest caz, sistemul de operare poate transfera pagini din RAM în

spațiul de swap, ceea ce permite continuarea funcționării proceselor fără blocare sau întrerupere.

Când are loc swap in și swap out?

Swap in are loc atunci când una sau mai multe pagini ale unui proces sunt mutate din spațiul de swap în memoria principală.
Swap out are loc atunci când una sau mai multe pagini ale unui proces sunt mutate din memoria principală în spațiul de swap.

Care este rolul unui page fault? În ce condiții apare?

Un page fault apare atunci când un proces încearcă să acceseze o pagină de memorie care nu se află în memoria principală, ci în spațiul de swap sau nu există încă. Rolul page fault-ului este de a gestiona accesul la pagină prin aducerea paginii în memoria principală (swap in) sau de a crea o nouă pagina pentru a fi utilizată de proces (în cazul în care nu exista deja). Acest mecanism de gestionare a memoriei permite unui proces să acceseze un spațiu mai mare de memorie decât cel disponibil în memoria principală și, în același timp, să împartă resursele de memorie între mai multe procese în mod eficient.

Care sunt secțiunile/zonele din spațiul de adrese al unui proces?

Spațiul de adrese al unui proces poate fi divizat în următoarele zone sau secțiuni:

- Zona de cod (.text): conține codul executabil al programului.
- Zona de date (.data): conține variabilele statice și constantele alocate în timpul compilării.
- Zona read-only (.rodata): conține date constante care nu pot fi modificate, cum ar fi string-urile.
- Zona BSS (.bss): conține variabilele alocate dinamic la execuție, dar care nu au fost inițializate.
- Stiva (stack): conține datele temporare, cum ar fi parametrii și variabilele locale ale apelurilor de funcții.
- Heap: conține obiectele alocate dinamic și poate fi folosit pentru a gestiona alocări de memorie în timpul execuției.

Ce secțiuni ale unui executabil se pot inspecta doar în timpul rulării?

Secțiunile .bss și heap-ul se pot inspecta doar în timpul rulării.

Care sunt zonele writable din spațiul de adrese al unui proces?

Zona .data și heap-ul sunt zonele writable din spațiul de adrese al unui proces.

De ce sunt avantajoase bibliotecile dinamice pentru spațiul de adrese al unui proces?

Bibliotecile dinamice sunt avantajoase pentru spațiul de adrese al unui proces deoarece:

- Îmbunătățesc gestionarea memoriei prin împărțirea funcționalității programului în mai multe biblioteci separate care pot fi încărcate la nevoie.
- Permit încărcarea bibliotecilor comune de mai multe procese, economisind astfel spațiu în memoria principală.
- Permit actualizarea bibliotecilor fără necesitatea recompilării programelor care le folosesc.

Două procese sunt pornite din același executabil, ce zone din spațiul de adrese vor partaja?

Două procese pornite din același executabil partajează zonele ".text" (codul) și "rodata" (datele read-only). Toate celelalte zone, cum ar fi ".data", ".bss" și zonele de heap sau stack, sunt separate pentru fiecare proces.

Se alocă un buffer a[100]. De ce a[105] NU va rezulta, în general, în Segmentation fault?

Accesarea elementului a[105] poate sau nu sa rezulte în Segmentation fault, în funcție de implementarea sistemului de operare și dacă adresa a[105] se afla în afara limitelor zonei de date a procesului. În unele cazuri, accesarea adresei poate duce la accesarea unei zone nealocate a memoriei, rezultand segmentation fault, în timp ce în alte cazuri, sistemul poate aloca automat memorie suplimentară, fără a declanșa eroarea.

În ce situație a[300] rezultă în Segmentation fault?

a[300] poate rezulta în Segmentation fault dacă acesta încearcă să acceseze un spațiu de adresă din afara zonelor alocate de sistemul de operare pentru proces. Dacă zona alocată inițial pentru acest buffer este limitată la a[100], accesarea a[300] poate încălca această limită, cauzând un Segmentation fault.

Câte pagini ocupă char a[100] pentru sistem standard cu pagini de 4096 de octeți?

Un buffer a[100] va ocupa 1 pagina pentru sistem standard cu pagini de 4096 octeți.

Câte pagini fizice alocă un apel mmap() care alocă 1MB? O pagină ocupă 4KB.

Un apel mmap() care alocă 1MB va aloca 256 pagini fizice (1 MB = 1024 KB = 256 x 4 KB).

Câte pagini fizice alocă un apel calloc() care alocă 1MB? O pagină ocupă 4KB.

Un apel calloc() care alocă 1MB va aloca 256 pagini fizice, fiecare pagina avand dimensiunea de 4KB.

Ce înseamnă maparea unui fișier în memorie? De ce este avantajos să mapăm fișiere față de folosirea read/write?

Maparea unui fișier în memorie înseamnă să se atribuie o secțiune din spațiul de adrese al unui proces unui fișier de pe disc, astfel încât fișierul să poată fi accesat ca un array simplu în memorie.
Avantajul acestei abordări este că sistemul de fișiere poate gestiona încărcarea și descărcarea automată a paginilor din fișier în memoria RAM, în funcție de necesitățile procesului, în timp ce citirile și scrierile din fișier pot fi realizate rapid, fără overhead-ul asociat cu utilizarea read/write. De asemenea, maparea fișierelor permite proceselor să comunice prin intermediul fișierelor, fără a fi nevoie de mecanisme de sincronizare complicate.

Câte page fault-uri se pot obține în cazul operației *a = b?

În cazul operației *a = b, se poate obține cel mult un page fault, dacă adresa de memorie la care se face atribuirea (pointer-ul a) nu este încă în pagina încărcată în memoria fizică. Acest page fault va fi tratat prin încărcarea paginii care conține această adresă în memoria fizică. În alte cazuri, unde adresa de memorie este deja încărcată în memoria fizică, nu se vor produce page fault-uri.

Care este numărul maxim de page fault-uri pe care îl poate genera expresia a = b + c?

Numărul maxim de page fault-uri care poate fi generat de expresia "a = b + c" este 1. Acest lucru se poate întâmpla în cazul în care una sau ambele variabile b și c sunt stocate într-o pagină ce nu este încă în RAM, astfel încât se generează un page fault. După încărcarea paginii în RAM, adunarea poate fi efectuată fără alte page fault-uri.

Ce informații sunt reținute în stivă? Ce variabile C?

Informațiile reținute în stivă (stack) sunt informații despre apelurile curente ale funcțiilor, incluzând adresele de întoarcere, argumentele funcțiilor, informații despre variabile locale.
În C, variabilele locale ale unei funcții sunt stocate în stivă. Acestea includ variabile temporare, variabile locale ale blocurilor de cod și variabile locale ale funcțiilor recursive. În plus, informațiile de pe stivă conțin și informații despre parametrii transmiși către funcție și despre valorile variabilelor globale care sunt accesate în interiorul funcției.

Ce se întâmplă la faza de loading (încărcarea unui executabil în memorie și crearea unui proces)?

La faza de loading, un executabil este încărcat în memorie și se alocă un spațiu de adrese pentru un proces. Procesul este creat prin apelarea sistemului de operare, care:

- Alocă zona text, zona data și zona stack a spațiului de adrese al procesului
- Încarcă fișierul executabil în zona text și, eventual, alte biblioteci dinamice în spațiul de adrese
- Inițializează zona data și variabilele statice
- Setează pointer-ul stivei și începe execuția codului din zona text, la adresa program counter-ului.

Ce înseamnă deturnarea fluxului de execuție a unui program (control flow hijack)? De ce este acest lucru relevant pentru un atacator?

Deturnarea fluxului de execuție a unui program (Control Flow Hijack) se referă la modificarea ordinii de execuție a codului într-un program. Un atacator poate să profite de acest lucru prin redirecționarea fluxului către cod malițios, care poate fi folosit pentru a accesa date sensibile sau a distruge informații, a crea backdoor-uri sau a compromite integritatea sistemului. Este relevant pentru un atacator deoarece permite acestuia să controleze un sistem sau să obțină acces la informații sensibile.

Ce înseamnă memory leak / memory disclosure? De ce este acest lucru relevant pentru un atacator?

Memory leak se referă la pierderea de memorie în cadrul unui program, unde aceasta nu mai poate fi accesată sau eliberată de către program. Acest lucru poate duce la epuizarea memoriei sistemului și căderea programului. Memory disclosure se referă la expunerea accidentală a informațiilor sensibile din memorie, precum parole, chei sau informații confidențiale.
Acest lucru este relevant pentru un atacator, deoarece el poate exploata vulnerabilitățile legate de memorie pentru a obține acces la informații sensibile sau pentru a compromite securitatea sistemului prin cauzarea unui comportament nedorit.

De ce în general, preferăm o împărțire a spațiului virtual de adrese între kernel space și user space? Și nu un spațiu dedicat pentru kernel space?

Preferăm o împărțire a spațiului virtual de adrese între kernel space și user space pentru a oferi un model de securitate mai robust și controlat. Aceasta separă și protejează zonele critice ale sistemului, cum ar fi structurile de date interne ale kernel-ului de accesul neautorizat din partea aplicațiilor utilizator. Acest lucru minimizează riscul de escaladare a privilegiilor și a altor atacuri de tip buffer overflow sau denial-of-service.

Ce secvență de cod C va duce la o excepție de acces la memorie (de tip Segmentation fault)? De ce?

O secvență de cod C care accesează o locație de memorie nealocată sau protejată la citire/scriere, cum ar fi:
int *p = NULL;
*p = 42;
va duce la o excepție de acces la memorie (Segmentation fault), deoarece pointer-ul p nu este inițializat cu o adresă validă și încercăm să accesăm o locație de memorie prin intermediul acestuia.

Cu ce diferă o funcție de o variabilă într-un executabil și/sau în cadrul spațiului de adrese al unui proces?

O funcție este un segment de cod care poate fi apelat de alte părți ale programului pentru a executa o anumită acțiune. O variabilă este o zonă de memorie în care se stochează o valoare. Funcțiile sunt poziționate în spațiul de adrese al unui proces ca fiind read-only, în timp ce variabilele pot fi read/write. Funcțiile sunt încărcate în memoria principală la începutul procesului și pot fi apelate din diferite locații în program, în timp ce valorile variabilelor pot fi modificate în timpul execuției programului.

Două procese partajează o zonă de memorie. Cum se manifestă acest lucru în tabelele de pagini ale celor două procese?

În acest caz, în tabelele de pagini ale celor două procese, acea zonă de memorie va avea aceeași adresă virtuală și va fi marcată ca fiind partajată între procese. Modificările făcute de un proces la acea zonă de memorie vor fi vizibile și pentru celălalt proces, iar acest lucru se datorează partajării memoriei.

Putem avea mai multă memorie fizică decât dimensiunea maximă a spațiului virtual de adrese al unui proces? Dar invers?

Da, putem avea mai multă memorie fizică decât dimensiunea maximă a spațiului virtual de adrese al unui proces, dar nu invers. Spațiul virtual de adrese poate fi mai mare decât cantitatea de memorie fizică disponibilă, dar în acest caz, OS-ul va folosi scheme de paginare și swap pentru a mapa părți din spațiul virtual de adrese pe memorie fizică.

Ce zone de memorie se aloca static? Dar dinamic?

Zonele de memorie alocate static sunt alocate la compilare și rețin valorile lor pe tot parcursul execuției programului. Exemple: variabilele globale și static, array-urile alocate static, structuri și enumerări.
Zonele de memorie alocate dinamic sunt alocate în timpul execuției programului folosind apeluri la funcții precum malloc(), calloc(), realloc() sau mmap(). Acestea pot fi libere sau realocate în funcție de nevoile programului.

Ce se întâmplă cu o variabilă modificată într-un proces copil din perspectiva procesului părinte?

Modificarea unei variabile într-un proces copil nu va afecta valoarea variabilei în procesul părinte. Procesele copil și părinte au fiecare propriul lor spațiu de adrese virtual și, prin urmare, o variabilă modificată într-un proces copil nu va modifica valoarea variabilei în procesul părinte.

Ce reprezintă un loader? Ce rol are acesta?

Un loader reprezintă un program care încarcă un alt program sau un sistem de operare în memoria principală a unui calculator. Rolul acestuia este de a aloca memorie şi de a începe execuţia programului sau a sistemului de operare încărcat.

La ce folosim apelul mprotect()? La ce mecanism de securitate putem face bypass folosind acest apel?

Apelul mprotect() din sistemul de operare se folosește pentru a modifica permisiunile de acces la o zonă de memorie alocată prin mmap(). Acesta poate fi utilizat pentru a schimba permisiunile de acces la o anumită zonă de memorie (read-only sau read-write).
Mecanismul de securitate care poate fi ocolit prin utilizarea acestui apel este ASLR (Address Space Layout Randomization). ASLR aranjează aleatoriu poziționarea codului executabil în memoria principală a unui sistem, cu scopul de a împiedica exploatarea vulnerabilităților de securitate prin injectarea codului malițios. Dacă un atacator reușește să modifice permisiunile

de acces la o zonă de memorie protejată prin ASLR (read-only sau read-write), poate să execute cod malițios în acel spațiu și să ocolească astfel mecanismul de securitate.

În ce zonă de memorie se află o variabilă globală, inițializată cu valoarea 0?

O variabilă globală inițializată cu valoarea 0 se află în zonă de memorie data. Această zonă conține variabilele globale și statice care sunt alocate la începutul programului și au valori cunoscute la compilare. Variabilele globale sunt accesibile din orice funcție din program și persistă pe tot parcursul execuției programului.

Calcul

De ce este nevoie de procese?

Sistemele de operare au nevoie de procese pentru a gestiona resursele sistemului (cum ar fi CPU, memorie, fișiere și periferice) și pentru a asigura o utilizare eficientă și sigură a acestora de către aplicații și utilizatori. Procesele împart resursele sistemului între aplicații și le permit să ruleze în paralel, fără a interfera una cu cealaltă. De asemenea, procesele permit sistemului să gestioneze problemele de securitate și performanță, precum și să asigure fiabilitatea sistemului prin recuperarea automată după erori.

De ce este nevoie de thread-uri?

Thread-urile sunt necesare pentru a îmbunătăți performanța și eficiența programelor prin paralelism și împărțirea sarcinilor în unități mai mici. Thread-urile permit programelor să ruleze mai multe operații simultan, îmbunătățind astfel viteza de execuție și utilizarea resurselor sistemului (cum ar fi CPU și memorie). De asemenea, thread-urile fac posibilă gestionarea mai ușoară a operațiilor care pot fi blocate, precum și creșterea flexibilității și a responsivității programului în general.

Ce este un proces?

Un proces este un program în execuție. Procesele sunt unitatea primitivă prin care sistemul de operare alocă resurse utilizatorilor. Orice proces are un spațiu de adrese și unul sau mai multe fire de execuție. Putem avea mai multe procese ce execută același program, dar oricare două procese sunt complet independente.

Ce este un thread?

Fir de execuție (sau thread) este unitatea elementară de planificare într-un sistem. Ca și procesele, firele de execuție reprezintă un mecanism prin care un calculator poate să ruleze mai multe task-uri simultan.

Cu ce diferă un thread de un proces?

Dacă un proces intampina o problema, nu afectează celelalte procese. Dacă un thread crapă, tot programul crapă.
Threadurile sunt mai eficiente decat procesele ca și timp de creare și schimbări de context.
Threadurile în general au nevoie de sincronizare explicită.

Cum este afectat spațiul virtual de adrese al unui proces în momentul creării unui thread?

Se mai aloca memorie pentru stiva noului thread (standard 8MB, dar se poate modifica).

Ce zone de memorie au comune thread-urile unui proces și ce zone au specifice?

Thread-urile unui proces au acces la aceleași zone de memorie ale procesului, cum ar fi heap și data. Acestea sunt zonele de memorie comune pentru toate thread-urile din același proces.
În schimb, fiecare thread are propriile sale zone de memorie specifice, cum ar fi regiștrii și stiva, care sunt utilizate pentru stocarea informațiilor specifice pentru fiecare thread. Aceste zone de memorie specifice permit fiecărui thread să aibă propriul său set de informații, cum ar fi variabilele locale și datele de stivă, fără a interfera cu alte thread-uri din același proces.

Ce conține PCB (Process Control Block)?

PCB (Process Control Block) este un obiect de sistem de operare care conține informații despre un proces. Conținutul standard al unui PCB poate include: id-ul procesului, prioritatea procesului, program counter, regiștrii, starea procesului, tabele de resurse și informații despre program.

Care sunt stările în care se poate găsi un thread?

Un thread poate fi în următoarele stări: running, ready, blocked, terminated.

Ce efect are apelul fork()?

Apelul fork() din sistemul de operare creează un proces copil asemănător cu procesul părinte. După apelare, există două procese separate care rulează în paralel, unul fiind procesul părinte și celălalt fiind procesul copil.

Ce resurse partajează/nu partajează procesul părinte și procesul copil în cazul apelului fork()?

Procesul părinte și procesul copil creat prin apelul fork() împart următoarele resurse: fișierele deschise, variabilele globale, signal handlers.
Procesul părinte și procesul copil au următoarele resurse separate: PC (Program Counter), registrii CPU, stack, PID (Process Identifier), data, file descriptors.

Cum modifica fork() si exec() spațiul virtual de adrese?

Funcția fork() creează un proces copil care are același spațiu virtual de adrese ca și procesul părinte.
Funcția exec() înlocuiește spațiul virtual de adrese al procesului curent cu cel specificat în apelul exec(), astfel încât procesul să execute noul program.

Ce efect are apelul exec()?

Exec() încarcă imaginea procesului copil dintr-un executabil in procesul curent. Loaderul încarcă imaginea in memorie și întoarce un pointer la imagine la kernel.

Câte threaduri se pot găsi în starea RUNNING, READY și WAITING?

Numărul de thread-uri în stările RUNNING, READY și WAITING variază în funcție de implementarea sistemului de operare și de aplicație. În general, un număr variabil de thread-uri pot fi în starea RUNNING în funcție de resursele disponibile, un număr variabil de thread-uri pot fi în starea READY să fie planificate pentru a rulea și un număr variabil de thread-uri pot fi în starea WAITING în așteptarea unei resurse sau a unei operații care să se completeze.

Ce este o schimbare de context? Ce se întâmplă la o schimbare de context?

Se referă la schimbarea unui proces care rulează pe un procesor (este în starea RUNNING) cu un alt proces (aflat în starea READY). Este necesară pentru a asigura folosirea optimă a procesorului (dacă un proces se blochează îi ia altul locul) și pentru asigurarea echității (fairness) a sistemului (procesele se schimbă cu altele pentru a permite câtor mai multe să ruleze în sistem).

Ce cauzează schimbări de context?

5 condiții principale care iti genereaza context switch:
1) RUNNING → READY - schimbare voluntara (yield) Yield înseamnă sa plece singur de pe procesor
2) RUNNING → BLOCKED (waiting) - schimbare voluntara, o acțiune executată explicit de procesor care are nevoie de resurse (operații blocante sau I/O o declanșează)
3) RUNNING → READY - schimbare nevoluntara (apare un proces candidat mai bun, prioritar)
4) RUNNING → READY - schimbare nevoluntare (eg expira cuanta)
5) RUNNING → TERMINATED - aici depinde ce fel de schimbare e după cum s-a terminat. Este scos de pe procesor.

Procesele I/O intensive din running vor avea tendința sa tranziteze în BLOCKED iar cele CPU intensive în READY.

Ce este o schimbare de context voluntară și o schimbare de context nevoluntară?

Specificat la întrebarea de mai sus.

Ce sunt threadurile cu implementare user-level și thread-urile cu implementare kernel-level?

Thread-urile cu implementare user-level sunt gestionate de aplicație în sine, în timp ce thread-urile cu implementare kernel-level sunt gestionate direct de sistemul de operare. User-level threads sunt mai ușor de implementat, dar pot fi limitate de contextul de blocare al sistemului de operare, în timp ce kernel-level threads sunt mai performante, dar mai greu de gestionat din cauza complexității implementării.

În ce situație este utilă zona TLS (thread local storage)?

Cand dorim sa avem variabile globale, astfel incat sa fie accesate de toate functiile, dar nu dorim ca acele variabile să fie partajate intre threaduri.

De ce este necesară sincronizarea proceselor/thread-urilor?

Pentru a ne asigura ca nu exista race condition sau situații în cazul în care programul are caracter nedeterminist.

Ce înseamnă race condition?

Evenimentul care are loc atunci cand 2 sau mai multe threaduri incearca sa acceseze aceeasi resursa comuna si sa o modifice simultan. Exemplu: cu a = 0, facem a++ care poate sa rezulte în a = 1 sau a = 2.

Ce înseamnă deadlock?

Cand 2 sau mai multe threaduri așteaptă simultan unul după altul.

Care sunt dezavantajele sincronizării?

Dezavantajele sincronizării în programare sunt: performanța scăzută, debugging dificil, complexitate crescută, blocarea proceselor, deadlocks.

Ce înseamnă TOCTOU (time of check to time of use)?

TOCTOU (Time-of-Check to Time-of-Use) se referă la o vulnerabilitate de securitate care apare atunci când un program verifică dacă o condiție este îndeplinită, dar între momentul verificării și momentul utilizării, condiția poate fi modificată de alt proces.

Când se blochează un producător în problema producator-consumator? Dar un consumator?

Producătorul se blocheaza cand bufferul e plin și așteaptă eliberarea bufferului pentru a putea produce în continuare.
Consumatorul se blochează atunci cand bufferul este gol. Va fi trezit cand a sosit un produs în buffer.

De ce este necesară prezența unei instrucțiuni de tipul atomic_compare_and_swap în fiecare ISA?

Instrucțiunea atomic_compare_and_swap este necesară în fiecare ISA (set de instrucțiuni arhitecturale) deoarece aceasta asigură o operare atomică de modificare a unei valori în memorie. În multithreading, mai multe threaduri pot accesa aceeași zonă de memorie, iar modificarea ne-atomă a acestei locații poate duce la probleme de concurență, cum ar fi race conditions.

Cu ce diferă un spinlock de un mutex? Când folosim spinlock-uri? Când folosim mutex-uri?

Spinlock-ul folosește busy-waiting și are operații de lock() și unlock() ieftine prin comparație cu mutex-ul. Operațiilor de lock() și unlock() pe mutex sunt de obicei costisitoare întrucât pot ajunge să invoce planificatorul. Având operații rapide, spinlock-ul este potrivit pe secțiuni critice de mici dimensiuni în care nu se fac operații blocante; în aceste cazuri faptul că face busy-waiting nu contează așa de mult pentru că va intra rapid în regiunea critică. Dacă am folosi un mutex pentru o regiune critică mică, atunci overhead-ul cauzat de operațiile pe mutex ar fi relativ semnificativ față de timpul scurt petrecut în regiunea critică, rezultând în ineficiența folosirii timpului pe procesor.

Deoarece spinlock-urile folosesc busy waiting, sunt recomandate doar pentru secțiuni critice mici, ce se execută rapid.

Regiunile critice cu operații de I/O sunt, de obicei, lungi. De asemenea, operațiile I/O pot duce la blocarea thread-ului, caz în care nu poate fi folosit spinlock-ul.

Ce efect are folosirea operatorului & din shell în crearea unui proces?

În shell, operatorul & la sfârșitul unei comenzi indică sistemului să ruleze acea comandă în fundal, ca un proces separat. Shell-ul va continua să execute alte comenzi în paralel cu acea comandă, în loc să aștepte terminarea acesteia.

Ce este un proces zombie? Cum apare un proces zombie? Care este problema proceselor zombie?

Un proces zombie este un proces care a terminat execuția, dar persistă în sistemul de operare ca o intrare în tabela de procese, așteptând să fie curățat de către părintele său.
Un proces devine zombie atunci când părintele său nu își termină sarcina de a colecta informațiile despre starea procesului și de a notifica sistemul de operare că procesul a terminat execuția. Acest lucru poate fi cauzat de diverse probleme, cum ar fi blocarea sau întreruperea părintelui sau de probleme cu gestionarea resurselor sistemului.
Problema cu procesele zombie este că ele încă ocupă loc în tabela de procese a sistemului de operare și pot consuma resurse precum memorie sau descriptori de fișiere. În cazul în care numărul de procese zombie crește prea mult, acestea pot afecta performanța sistemului și pot cauza erori.

Ce este un proces orfan? De ce un proces este orfan foarte puțin timp? Poate fi un proces zombie orfan? Ce se întâmplă cu un proces zombie orfan?

Un proces orfan este un proces care a rămas fără un părinte. Un proces devine orfan atunci când părintele său este terminat sau întrerupt înainte de a colecta starea procesului copil.
Un proces este orfan foarte puțin timp pentru că sistemul de operare îi asignează un nou părinte, de obicei procesul init, care își va îndeplini responsabilitatea de a colecta starea procesului copil atunci când acesta se termină.
Un proces poate fi orfan și zombie în același timp, în cazul în care părintele său a fost terminat sau întrerupt înainte de a colecta starea procesului copil și procesul copil a terminat deja execuția. În acest caz, procesul zombie orfan va fi curățat de către procesul init, care își îndeplinește responsabilitatea de a colecta starea proceselor orfane.

Ce se întâmplă dacă un thread realizează un acces invalid la o zonă de memorie?

Primește SIGSEGV și se apelează rutina de tratare a excepției.

Poate un thread să acceseze stiva altui thread? Cum?

Trebuie ca threadurile sa fie ale aceluiași proces.

De ce schimbarea de context între două thread-uri ale aceluiași proces este mai rapidă decât schimbarea de context între două thread-uri din procese diferite?

Pentru ca doua thread-uri care aparțin aceluiași proces share-uiesc memoria. Nu se da flush la TLB.

Ce limitează numărul maxim de threaduri care pot fi create în cadrul unui proces?

Numărul maxim de thread-uri care pot fi create într-un proces este limitat de mai mulți factori, incluzând: sistemul de operare, resursele sistemului, arhitectura hardware, biblioteci și tehnologii de programare.

Ce limitează numărul maxim de procese care pot fi create în cadrul unui sistem de calcul?

Numărul maxim de procese care pot fi create în cadrul unui sistem de calcul este limitat de mai mulți factori, incluzând: sistemul de operare, resursele sistemului, arhitectura hardware.

Ce se întâmplă când toate procesele sistemului sunt blocate?

Pe fiecare procesor se planifica un proces special numit IDLE (care face busy waiting) dacă nu exista alt proces care poate fi planificat. (Instrucțiunea HLT din x86)

Ce înseamnă waiting time (timp de așteptare) în planificarea proceselor?

Noțiunea de waiting time se referă la timpul de așteptare al unui proces în coada READY a planificatorului. Pentru un sistem interactiv/responsiv este de dorit ca timpul de așteptare să fie cât mai scurt.

Putem avea un sistem multi-core cu un singur proces aflat în starea RUNNING și mai multe procese în READY?

Da, este posibil să avem un sistem multi-core cu un singur proces în starea RUNNING și mai multe procese în READY.
Într-un sistem multi-core, fiecare nucleu poate gestiona un proces diferit, astfel încât este posibil să aveți mai multe procese în starea READY care așteaptă să fie alocate unui nucleu și să fie puse în execuție. În același timp, un singur proces poate fi executat pe unul dintre nucleele disponibile.

Ce înseamnă starea READY/RUNNING/TERMINATED/WAITING(BLOCKED)?

Starea READY înseamnă că un proces este pregătit să ruleze, dar nu rulează încă din cauza alocării insuficiente a resurselor (cum ar fi procesorul) de către sistemul de operare.
Starea RUNNING înseamnă că un proces este în curs de execuție, utilizând unul sau mai multe resurse (cum ar fi procesorul).
Starea TERMINATED înseamnă că un proces a încetat să mai ruleze, fie din cauza finalizării normale a executării sale, fie din cauza unei erori sau a unei întreruperi externe.
Starea WAITING (BLOCKED) înseamnă că un proces este într-o pauză voluntară, așteptând un eveniment sau o resursă necesară pentru a continua executarea. În această stare, un proces nu rulează și nu este pregătit să ruleze până când evenimentul sau resursa necesară nu devin disponibile.

Ce este un proces I/O intensive?

Un proces care lucrează mult cu I/O-ul și in consecinta are multe apeluri blocante (trebuie de multe ori sa astepte pentru a primi și a scrie date). De cele mai multe ori un astfel de proces cand va ieși din starea RUNNING se va muta în BLOCKED.

Ce este un proces CPU intensive?

Un proces care face calcule intense, gen machine learning și care nu se blochează foarte des, pentru a ieși din starea RUNNING deobicei e nevoie sa fie scos nevoluntar. De cele mai multe ori un astfel de proces cand va ieși din starea RUNNING se va muta în READY.

Cum tratează planificatorul procesele I/O intensive și procesele CPU intensive?

Planificatorul tratează diferit procesele I/O intensive și procesele CPU intensive în funcție de strategia de planificare utilizată. Unele strategii de planificare, cum ar fi planificarea cu priorități, dau prioritate proceselor care au nevoie de I/O frecvent și rapid, pentru a reduce timpul de așteptare al acestora. Alte strategii de planificare, cum ar fi planificarea cu rotire (round-robin), asigură o utilizare echitabilă a resurselor CPU pentru toate procesele, incluzând procesele CPU intensive.

Două thread-uri ale unui proces execută aceeași funcție. Care sunt diferențele între cele două thread-uri?

Cele două thread-uri ale unui proces pot avea diferențe în următoarele aspecte: contextul de execuție, timp de execuție, ID-ul thread-ului.

Cu ce diferă procesul copil (creat prin fork) de procesul părinte?

Procesul copil creat prin funcția "fork" diferă de procesul părinte în următoarele moduri: ID-ul procesului, adresa de memorie, resurse, execuție, variabile de mediu, prioritate.

Cu ce diferă un proces zombie de un proces orfan?

Procesele zombie ocupă resurse în sistem, în timp ce proceselor orfane le pot fi alocate noi părinți sau pot fi terminate în mod normal.

Ce parametri ai planificatorului trebuie să modificăm pentru a avea un sistem cu productivitate mai mare?

Parametrii planificatorului care pot fi modificați pentru a îmbunătăți productivitatea sistemului pot include: algoritmul de planificare, prioritatea proceselor, timp de quantum, parametrii de afișare.

Ce parametri ai planificatorului trebuie să modificăm pentru a avea un sistem cât mai interactiv (responsive)?

Pentru a avea un sistem mai interactiv, trebuie să modificați următorii parametri ai planificatorului: timpul de întârziere a task-urilor, prioritatea task-urilor, alocarea resurselor (procesor, memorie, etc.), frecvența de actualizare a stării sistemului, modul de gestionare a întreruperilor.

Am avea nevoie de folosirea unui apel de sistem pentru crearea unui thread în cazul unei implementări de tip user-level threads? Dar în cazul deschiderii unui fișier în același scenariu?

Da, în cazul unei implementări de tip user-level threads, se folosește un apel de sistem pentru a crea un nou thread.
În cazul deschiderii unui fișier, în același scenariu cu user-level threads, se folosește tot un apel de sistem, de exemplu, open().

Ce se întâmplă dacă folosim apeluri de sistem blocante în interiorul unui spinlock?

Dacă se folosesc apeluri de sistem blocante în interiorul unui spinlock, poate apărea un deadlock, deoarece spinlock-ul blochează accesul la resurse, iar apelul de sistem blocant va aștepta accesul la aceleași resurse, ceea ce duce la un blocaj reciproc. De asemenea, aceasta poate duce la un blocaj în sistem, deoarece alte procese sau thread-uri nu vor putea accesa resursele până când spinlock-ul nu va fi eliberat și apelul de sistem blocant nu se va termina. Prin urmare, se recomandă evitarea folosirii apelurilor de sistem blocante în interiorul unui spinlock.

De ce este necesară folosirea prefixului LOCK pentru realizarea operațiilor atomice?

Prefixul LOCK este necesar pentru a realiza operații atomice, deoarece acestea asigură că instrucțiunile specifice sunt executate în mod neîntrerupt, fără a fi interferate de alte procese sau thread-uri. Această funcționalitate poate fi utilizată pentru a preveni race condition-urile

și a garanta integritatea datelor în scenariile în care mai multe procese sau thread-uri accesează aceleași resurse în același timp.

Care este avantajul folosirii mutexurilor în defavoarea spinlockurilor și invers?

Avantajele folosirii mutexurilor în defavoarea spinlock-urilor sunt: prevenirea deadlock-urilor, gestionarea prioritizării, economie de energie.
Avantajele folosirii spinlock-urilor în defavoarea mutexurilor sunt: performanță, eficiență în utilizarea resurselor, simplitate.

Care este echivalentul apelului wait() pentru threaduri?

Echivalentul apelului wait() pentru thread-uri poate fi join(). Apelul join() permite unui thread principal să aștepte terminarea unui alt thread, astfel încât să se poată asigura că toate resursele alocate acestuia au fost eliberate înainte de încheierea programului.

De ce este nevoie de apelul pthread_wait() / pthread_join()?

Apelul pthread_join() / pthread_wait() este necesar pentru a aștepta terminarea unui thread. Acest apel permite unui thread principal să aștepte până când un alt thread s-a terminat și a eliberat toate resursele alocate, astfel încât să se poată evita pierderea de date sau erorile de memorie care pot rezulta din terminarea prematură a unui thread fără eliberarea completă a resurselor asociate acestuia.

Ce tranziție între stările unui thread nu este posibilă?

Nu este posibilă tranziția dintr-o stare terminată a unui thread în orice altă stare. Odată ce un thread a ajuns în starea TERMINATED, acesta nu mai poate fi reluat sau redat în altă stare. Acest lucru se datorează faptului că toate resursele alocate unui thread, cum ar fi stack-ul și alte structuri de date, pot fi eliberate de sistemul de operare și nu mai sunt disponibile pentru a fi folosite în viitor.

I/O

Ce conține un FCB (File Control Block)?

FCB-ul este o structura internă a sistemului de fișiere în care este păstrat state-ul unui open file. FCB-ul e manage-uit de kernel și rezidă în memoria programului care utilizează fișierul, nu în memoria SO-ului.

Ce reprezintă un descriptor de fișier?

Este un număr (întreg) ce referă o intrare în tabela de descriptori de fișier. Este folosit în operații de lucru cu fișiere, pentru a identifica un fișier deschis.

Ce reprezintă tabele de descriptori de fișiere?

Tabela de descriptori de fișier a unui proces conține pointeri; ca structură de date este un vector de pointeri. Acești pointeri referă structuri de fișier deschis de proces. Când un proces deschide un fișier, se alocă o structură de fișier deschis, iar adresa acestei structuri este stocată într-un loc liber (indicat de descriptorul de fișier) din tabela de descriptori de fișier.

Câte tabele de descriptori de fișiere se găsesc într-un sistem de operare?

Fiecare proces are o tabelă de descriptori de fișier, deci vor exista, la nivelul sistemului de operare, atâtea tabele de descriptori de fișier câte procese există în acel moment în sistem.

Ce efect are apelul dup()?

Apelul dup() creează un descriptor de fișier nou, care se referă la același fișier deschis ca și descriptorul de fișier original. Noul descriptor are un număr diferit și poate fi utilizat independent pentru a accesa același fișier. După apelarea funcției dup(), ambele descriptoare de fișier pot fi utilizate simultan și modificările efectuate prin unul vor fi reflectate prin celălalt.

Ce efect are apelul close()?

Apelul close() închide descriptorul de fișier specificat și eliberează resursele asociate acestuia (cum ar fi descriptorii de fișier sau buffer-ul asociat). După închidere, descriptorul nu poate fi folosit pentru a accesa fișierul asociat. Acest apel poate fi utilizat pentru a închide un fișier deschis, eliberând astfel resursele sistemului alocate pentru acesta.

Ce apeluri modifică pointer-ul/cursorul de fișier (file pointer)?

1. lseek/fseek, apeluri al căror rol este de modificare a cursorului de fișier;
2. read/fread/fgets – la fiecare citire cursorul de fișier este incrementat cu numărul de octeți citiți; 3. write/fwrite/fputs/fprints – la fiecare scriere cursorul de fișier este incrementat cu numărul de octeți scrișI;
4. ftruncate – trunchiază fișierul (cursorul este plasat pe 0);

Ce apeluri modifică dimensiunea fișierului?

Apeluri care pot modifica dimensiunea unui fișier sunt write (poate scrie dincolo de limita unui fișier), ftruncate (modifică chiar câmpul dimensiune) sau open cu argumentul O_TRUNC care reduce dimensiunea fișierului la 0.

Ce efect are apelul/comanda truncate?

Trunchiaza fișierul la o dimensiune specificata. Fișierul poate fi trunchiat la 0 și prin apelul open() cu O_TRUNC.

Care sunt tipurile de fișiere pe un sistem de fișiere uzual Unix?

Regular, directory, symbolic link, FIFO special, block special, character special si socket.

Ce este un sistem de fișiere virtual?

Un sistem de fișiere virtual este un software care permite utilizatorului să creeze și să gestioneze fișiere și directoare pe un sistem de operare virtual, care nu este necesar să fie fizic prezent pe dispozitivul utilizatorului. Acesta poate fi utilizat pentru a accesa fișierele într-un mod similar cu accesarea fișierelor pe un sistem de operare fizic, dar fără limitările spațiului de stocare sau performanța hardware-ului.

Ce este un dispozitiv virtual?

Un dispozitiv virtual este un software care simulează funcționarea unui dispozitiv hardware real, permițând utilizatorului să interacționeze cu acesta într-un mod similar cu interacțiunea cu un dispozitiv hardware real. Acesta poate fi utilizat pentru a înlocui necesitatea unui dispozitiv hardware real, pentru a crea un mediu de testare pentru aplicații sau pentru a accesa un sistem de operare virtual.

Ce tipuri de dispozitive cunoașteți? Clasificați-le din orice punct de vedere cunoașteți

UNIX putem distinge două tipuri de device-uri:
1.Dispozitive caracter(aka serial devices): se ocupă cu gestionarea datelor ‘one character at a time’.
Exemple de dispozitive caracter sunt terminalele, tastaturile, mouse-uri, placi de sunet etc. Datele pot fi scrise sau citite o singura data.
În situația în care un byte e citit de la tastatura/mouse, același byte nu poate fi citit de alt program.
2.Dispozitive block(aka random access devices): sunt responsabile cu gestionarea datelor ‘one block at a time’. Dimensiunea unui block este în general un multiplu de 512 exprimat în bytes(poate varia în funcție de tipul dispozitivului, în general avem 1024) și avem drept exemplu diverse medii de stocare precum disk-uri sau casete(d-alea cu banda magnetica).
Aici datele sunt persistente și exprimate în sectiuni contigue și spre deosebire de dispozitivele de tip caracter, putem citi/scrie datele de mai multe ori.
Exemple de device name-uri:
/dev/hda, /dev/hdb, /dev/mouse, /dev/modem, /dev/zero. /dev/null, /dev/tty(consola)
Note: Toate dispozitivele sunt reprezentate sub formă de fișier(în Unix și Linux).

Cu ce diferă un dispozitiv de tip bloc de un dispozitiv de tip caracter? Dați câte un exemplu de fiecare.

Dispozitiv tic caracter: unseekable(explicație mai jos), n-are nevoie de buffering, accesul în cadrul acestui tip de device se face sub forma de stream ‘one byte at a time’, driverul asociat este mult mai simplu, necesita mai puțin efort fiindcă are de gestionat o singura pozitie(cea curentă). Apelurile read(), write() sunt blocante.
Dispozitiv tip block: E accesat printr-un cache, deci avem nevoie de un buffer, driver-ul asociat kernel-ului necesita mai multă atenție/efort având responsabilitatea de a naviga printre chunks of blocks în cadrul mediului de stocare. Apelurile bread(), bwrite() pot fi asincrone.

De ce nu are sens operația de seek pe un dispozitiv de tip caracter?

Operația de seek n-are sens pentru un dispozitiv de tip caracter deoarece tipul asta de device are o singura pozitie, cea curentă(fiindcă datele vin sub forma de stream ‘one character at a time’), așadar nu putem sa miscam un pointer în fața sau înapoi în fișier.

Ce adresă IP locală și ce port local are un socket întors de apelul accept()?

Funcția returnează file descriptor-ul socket-ului nou creat, care contine adresa/port-ul asociat peer-ului (aka aplicația careia îi permitem sa se conecteze).

Ce valoare poate întoarce un apel read() sau un apel write()?

Un apel de sistem read() sau write() întoarce numărul de bytes citiți/scrisi sau 0 în cazul end-of-file la read().

Ce operații se pot face pe fișiere?

Operațiile care sunt definite în structura de file_operations din cadrul kernelului, din care se pot aminti cateva mai populare: read(), llseek(), poll(), ioctl(), mmap(), write() , open(), close() etc.

Ce operații asupra fișierelor modifică/nu modifică valoarea cursorului unui fișier?

Nu modifica cursorul: operațiile de schimbare ale atributelor unui fișier, operațiunile de open/close/create.
Modifica cursorul: operațiile de poziționare într-un fișier, scrierea/citirea și trunchierea.

Ce operații asupra fișierelor modifică/nu modifică dimensiunea fișierului?

Modifica: write dacă scrii peste dimensiunea curentă, truncate
Nu modifică: lseek, read

Unde este reținută valoarea cursorului de fișiere (file pointer) și unde este reținută dimensiunea fișierului?

Dimensiunea fișierului e retinuta in structura inodului(pe langa owner id, group id, flaguri de sistem/user, device id etc), iar file pointer-ul rezidă în cadrul kernel-ului, acesta avand rolul de I/O managing in File Descriptor Table. (le găsim de obicei la /proc/__PID__/fd).

De ce avem două buffere asociate fiecărui socket, ce rol are fiecare?

Un buffer e de send, altul de receive.

Ce este o operație asincronă?

Operațiile asincrone au drept caracteristică faptul că pot fi executate în alta ordine fata de cea în care apar în cod.

Ce este o operație neblocantă?

O operație neblocanta permite executarea altor operații, chiar dacă încă nu a ajuns la finalul execuției.

Ce întoarce o operație asincronă?

O operație asincronă poate returna un obiect Promise sau un valoare directă în funcție de implementare. Promise reprezintă un obiect care se va rezolva sau va rejeta într-un viitor incert, și poate fi folosit pentru a gestiona rezultatul unei operații asincrone. Valoarea directă poate fi returnată atunci când operația asincronă se termină și are un rezultat disponibil imediat.

Ce întoarce o operație blocantă?

O operație blocantă returnează un rezultat imediat, după ce procesul s-a terminat. În timp ce o aplicație execută o operație blocantă, întreaga aplicație este blocată până când acea operație se termină și returnează rezultatul.

Cu ce diferă un socket de rețea de un socket UNIX?

Socketii UNIX sunt mecanisme de IPC(interprocess communication) și permit schimb de date bidirecțional între procese care rulează în același sistem, în timp de socketii de retea sunt utili in cazul comunicatiei intre procese situate in retele/sisteme diferite.

Care este diferența între un pipe anonim și un pipe cu nume (named pipe)?

Pipe-ul anonim nu are un nume și este accesibil prin intermediul a doi file descriptori generati de funcția pipe(), în timp ce un pipe cu nume primește numele de la user prin care este referentiat de către reader/writer. Un named pipe poate avea mai multe procese care pot comunica prin acesta, dar unul anonim permite numai comunicarea unidirectionala intre un proces părinte și unul copil.

Ce este buffer cache-ul? Care este rolul său?

La lucrul cu fișiere pe disc, apelurile read/write nu accesează direct spațiul de pe disc, ci copiaza date dintr-un buffer din user-space într-unul din kernel-space denumit ‘buffer cache’. Scopul este de a nu genera timpi morți cauzati de operații repetate și încete pe disc.

De ce operația write pe fișiere este foarte rar blocantă?

E rar blocanta fiindcă procesul de copiere a conținutului buffer-ului din user-space în kernel-space durează foarte puțin. Cazuri în care devine blocanta sunt rare, precum atunci cand nu exista pagini libere și unele trebuie sa fie eliberate și cazul în care write-ul are ca target mijlocul unui fișier, ceea ce presupune citirea conținutului din jurul poziției.

În ce situație operația read() pe fișier se blochează?

Operația read() pe fișier poate fi blocantă în următoarele situații:

- Când fișierul este blocat pentru scriere de o altă aplicație sau proces, astfel încât read() nu poate accesa datele din fișier.
- Când fișierul este aflat pe un sistem de fișiere cu viteză lentă, cum ar fi un dispozitiv de stocare mecanic, read() poate fi blocat până când datele sunt disponibile.
- Când fișierul este aflat pe un sistem de fișiere care nu suportă operații asincrone, cum ar fi un sistem de fișiere cu un singur nivel, read() poate fi blocat până când datele sunt disponibile.

Care este rolul unui device driver?

Rolul sau este de a permite unui device hardware să comunice cu sistemul de operare.(Ex: transfer de date)

Ce rol are controller-ul hardware?

Administrează corespunzător un periferic (cunoaște specificitățile perifericului respectiv).

Ce înseamnă zero-copy? Ce mecanism/apel folosește zero-copy?

Zero-copy presupune transferul unei informații între 2 buffere fără trecerea prin user space.
Mecanism/apel: splice(), sendfile()

Ce garanții ni se oferă în momentul în care apelul send() se întoarce în user-space?

Datele au fost copiate într-un send buffer.

Cu ce diferă afișarea folosind printf() față de folosirea write()?

Printf() poate interpreta formate diferite și poate afișa date într-un format specificat prin intermediul unui șir de formatare, în timp ce write() permite scrierea directă a datelor într-un descriptor de fișier. Aceasta nu permite formatarea sau interpretarea datelor și scrie date brute, fără a le formata într-un format specificat.

De ce subsistemul de networking nu folosește buffer cache-ul?

Subsistemul de networking nu folosește buffer cache-ul pentru a reduce overhead-ul și a asigura o transmisie mai rapidă a datelor.

Ce rol are apelul / comanda sync?

Apelul sau comanda sync are rolul de a forța sistemul de operare să scrie toate datele din buffer cache-ul pe pe disk.

Care este rolul apelului ioctl / DeviceIoControl?

Apelul ioctl are rolul de a controla și accesa informații despre dispozitivele din sistemul de operare.

De ce în general doar utilizatorul root are permisiuni de scriere (uneori doar root are permisiuni de citire) pe intrările din /dev?

Pentru ca accesul direct la hardware se face doar din modul privilegiat. Dacă oricine ar putea face modificări pe modul de funcționare ale device driverelor, atunci ar putea apărea probleme de securitatea, se pot crea vulnerabilitati si comunicarea cu dispozitivele de I/O ar fi nefuncțională. Dorim sa pastram intactă comunicarea cu aceste dispozitive. Se specifica faptul ca oricum procesorul prioritizează comunicarea cu Dispozitivele de I/O fără a exista interferențe cu ce scriu alte aplicații în memoria RAM.

De ce este apelul fwrite() mai rapid decât write atunci când facem multe scrieri?

Apelul fwrite() poate fi mai rapid decât apelul write în cazul în care sunt efectuate multe scrieri, deoarece fwrite() utilizează un buffer intermediar pentru a amortiza costul de acces la dispozitivul de stocare.

Ce se întâmplă dacă facem open de mai multe ori consecutiv pe același fișier?

Dacă se face open de mai multe ori consecutiv pentru același fișier, vor fi returnate mai multe identificatoare de fișiere diferite, fiecare referindu-se la același fișier.

Cum se modifica tabelul de file descriptori după open() și dup()? Este echivalent cu doua apeluri open()?

Dacă dau două open() sunt 2 instanțe diferite, separate.
dacă dau după ce se schimba în dup se schimba și în open, pentru ca pointeaza către același mod.

Ce fișiere sunt deschise, în general, la crearea unui proces nou?

Fișierul executabil, bibliotecile ce trebuie rulate pt proces.
Dacă am ls se deschide fișierul executabil la el și cam atat.

De ce este utilă prezența unor dispozitive pur virtuale în ierarhia /dev (ex. */dev/vboxnetctl*, */dev/urandom*)?

Pentru ca astfel comunica software-ul cu hardware-ul, și lasa aplicațiile sa interacționeze cu hardware-ul ca și cum sunt niste fisiere.

De ce, în general, apelul write() pe un fișier nu blochează threadul curent?

Nu blochează thread-ul curent pentru ca bufferul nu ramane des fără spațiu.

În ce situație un apel read() pe un fișier blochează threadul curent?

Cand este gol buffer-ul.

De ce spunem că operațiile read() / write() seamănă cu operația memcpy()?

Ambele manipulează data, dar diferenta principala e ca memcpy operează pe memorie și read()/write() operează pe fișiere sau device-uri.

Interacțiunea între aplicații

Ce forme de comunicare inter-proces cunoști?

Există mai multe forme de comunicare inter-proces în sistemele de operare, inclusiv:

- Pipe-uri: Permit transferul de date între procese prin intermediul unei canalizări anonime (unidirectionale sau bidirectionale).
- Semafoare: Permit proceselor să sincronizeze accesul la resurse comune prin intermediul unui mecanism de control al accesului.
- Mesaje: Permit transmiterea de informații între procese prin intermediul unui sistem de mesagerie care poate fi utilizat pentru a comunica între procese prin intermediul unui canal de comunicare.
- Memorii mapate de fișier: Permit proceselor să acceseze o zonă de memorie partajată, astfel încât să poată fi schimbate informații între procese.
- Sockets: Permit proceselor să transmită date între ele prin intermediul unui protocol de rețea, fie local sau remote.

Ce este un semnal? Când se trimite un semnal către un proces?

Notificarea asincrona pe care i-o trimite unui proces sau unui thread specific al sau. Semnalele sunt trimise către un proces prin intermediul unei apeluri de sistem sau prin intermediul unui eveniment generat de sistemul de operare

Cine trimite un semnal unui proces?

Sistemul de operare, un alt proces sau el însuși.

Cum este implementat operatorul | din shell?

Folosește dup2 în implementare pentru redirectare. Avem un pipe anonim(capăt citire și capăt scriere). Outputul primului proces este inputul celui de-al doilea. Primul proces scrie prin capătul de scriere al pipeului și cel de-al doilea citește prin cel de citire.

Care sunt avantajele și dezavantajele folosirii memoriei partajate pentru comunicarea inter-proces?

Avantaj: Mai puțină zonă de memorie folosită; mai rapidă comunicarea prin memorie decat prin mesaje/socketi
Dezavantaj: Sincronizarea, în lipsa ei se pot genera date incoerente, nu poți trimite pointeri.

Care sunt avantajele și dezavantajele pipe-urilor pentru comunicarea inter-proces?

Avantaje: comunicare între procese înrudite, dacă sunt cu nume poți și între procese total diferite. | e un mod super easy de redirectare în linia de comanda
Dezavantaje: comunicare unidirectionala; destul de basic(poti doar sa scrii si sa citesti). Nu poți mapa fișiere prin pipe în memorie. Au capacitate maximă per pipe, mai mica

Care este limitarea folosirii pipe-urilor anonime?

Sunt limitate numai la doua procese, au master și slave.

Care este diferența principală între FIFO și socket UNIX?

Socket-urile pot fi folosite pentru mai multe procese simultan (berkley asculta la mai multe rețele, unix asculta la un fisier din calculator).
FIFO poate fi folosit doar pentru 2 procese (oricare) simultan, socket-ul e mult mai whore.

Ce primitive de interacțiune pot fi folosite între procese de pe sisteme diferite?

Listen(), recv()

Ce primitive de comunicare pot fi folosite doar între threaduri?

Mutex, spinlock

Ce primitive de comunicare pot fi folosite doar între procese înrudite?

Pipe este cel mai des folosit pentru procese înrudite.

Cum pot două procese să folosească semafoare anonime?

Două procese inrudite pot folosi un semafor anonim creand un semafor cu comanda sem_init și să paseze adresa semaforului ca parametru pentru copil.

Cum pot două procese neînrudite să folosească semafoare anonime?

Două procese neinrudite pot folosi semafoare anonime creand semaforul în memorie shared (folosind mmap) și sa pastreze un pointer către acea regiune de memorie.

Ce rol are apelul mmap() în cazul partajării memoriei între două procese?

Este utilă pentru IPC (inter process communication) si evita overheadul ce era creat copiind date între procese și elimina nevoia de sincronizarea accesului la datele comune.

Ce mecanisme putem folosi pentru a asigura accesul exclusiv la o zonă de memorie partajată între două procese?

Folosim primitive de sincronizare, semafoare, mutex, spinlock, futex.

De ce este nevoie de sincronizare la folosirea memoriei partajate dar nu la folosirea unui socket pentru comunicarea între două procese?

Socketul face sincronizarea automată, fiind intre 2 sisteme diferite.

Ce înseamnă aplicație omogenă? Dați un exemplu.

O aplicație care are toate componentele create folosind aceeași tehnologie, facand-o simpla. Astfel de aplicații se folosesc acolo unde avem nevoie de un comportament simplu și bine definit.

Ce înseamnă aplicație eterogenă? Dați un exemplu.

O aplicație care combina și integrează componente create cu diferite tehnologii / limbaje de programare, și poate lua punctele tari ale fiecaruia, flexibil si interoperabil astfel de aplicații se folosesc în computing, servicii financiare și multimedia.

Care este avantajul și dezavantajul distribuirii unei aplicații pe mai multe sisteme?

Avantaj: e portabila
Dezavantaj : e mai complexă

Care este avantajul implementării unei aplicații în format multi-process față de format multi-threaded?

Format multi process - securitate, pentru fiecare tab ai cate un proces la chrome
Format multi-threaded - iti ofera mai puțin overhead, nu se da flush la TLB cand se face context switch

Cum pot două procese partaja memorie anonimă? Adică folosind flagul MAP_ANONYMOUS la mmap().

Folosind flagul de MAP_SHARED.

Ce mecanism de interacțiune între aplicații au un buffer și ce mecanisme de interacțiune nu au un buffer?

Au buffer: socket-urile
N-au buffer: semnalele